

Tracce Esercizi Thread

1. **ESERCIZIO_01.** Scrivere un programma C, secondo lo standard Posix, in ambiente Linux, che date due matrici A e B di dimensioni $n \times m$ di interi tra 0 e 255, ne determini una matrice C in cui ogni entrata $C(i,j)$ è ottenuta dalla divisione $A(i,j)/B(i,j)$. Il calcolo avviene concorrentemente usando m threads, uno per colonna. Appena un thread termina il calcolo della propria colonna di C, attende che tutti gli altri thread abbiano finito. A quel punto, un thread $n+1$ -esimo stamperà la matrice C. Usare i semafori Posix.
2. **ESERCIZIO_02.** Scrivere un programma C che crea M thread produttori e N thread consumatori (con $M = 2 \times N$), con M ed N presi come parametro da riga di comando. Tutti i thread condividono una variabile intera. Ogni thread produttore incrementa di due unità la variabile condivisa se e solo se il valore della stessa è inferiore a 100. Ogni thread consumatore decrementa di due unità la variabile condivisa se e solo se il valore della stessa è superiore a 25. Tutti i thread, dopo l'operazione sulla variabile condivisa (di incremento o di decremento) dormono per 2 secondi. Per implementare la sincronizzazione utilizzare i semafori Posix basati su nome.
3. **ESERCIZIO_03.** Scrivere un programma in C e Posix sotto Linux che utilizzi la libreria Pthread per risolvere il seguente problema: date due matrici A e B, di dimensioni $n \times n$ di numeri interi tra 1 e 100 generati casualmente, creare n thread dove l' i -esimo thread provvede a sommare le entrate delle righe i -esime delle due matrici in modo da ottenere una matrice Somma di dimensioni $n \times n$. Sincronizzare i thread in modo da sommare le righe delle due matrici A e B nell'ordine 1, 2, ..., n . Inoltre, un thread $n+1$ -esimo attende che la matrice Somma sia calcolata, per stamparne, successivamente, il contenuto. Usare semafori e variabili di condizione.
4. **ESERCIZIO_04.** Si consideri una matrice di caratteri con dimensioni $m \times n$, dove m ed n sono passati tramite riga di comando. La matrice è allocata dinamicamente. Scrivere un programma in C (POSIX, ambiente LINUX) che, dato un carattere x come argomento, avvii n thread che esplorino ciascuno una colonna alla ricerca del carattere x . Ogni thread, trovata un'occorrenza, aggiorna un contatore in un array chiamato trovati di dimensione n . Un thread aggiuntivo deve attendere la fine dell'elaborazione per poi stampare l'intero array trovati. Usare variabili di condizione.
5. **ESERCIZIO_05.** Scrivere un programma in C che (in ambiente Linux e utilizzando la libreria Pthread) crei 2 thread che eseguono la funzione "incrementa" che a sua volta accede alle variabili glob.a e glob.b di una struttura dati condivisa glob e ne incrementi il loro valore di 1 per 100 volte. Al termine, quando i thread avranno terminato con gli incrementi, il thread principale stamperà a video i valori delle variabili test.a e test.b. Per la gestione della sincronizzazione si utilizzino i mutex allocati dinamicamente.
6. **ESERCIZIO_06.** Data una matrice $n \times n$ (n pari) di interi generati casualmente, allocata dinamicamente, con n argomento da riga di comando, creare n thread che prelevano casualmente un elemento dalla riga di competenza (thread i -esimo, riga i -esima) e lo inseriscano concorrentemente in un vettore di $(n+1)/2$ elementi. Un thread $n+1$ -esimo attende il riempimento del vettore per stampare il contenuto dello stesso e per stampare il numero di elementi inseriti nel vettore da ciascun thread. Usare mutex e variabili di condizione.
7. **ESERCIZIO_07.** Si realizzi un programma in C e Posix sotto Linux che, con l'ausilio della libreria Pthread, lancia m thread per calcolare il prodotto di due matrici di dimensione $m \times n$ e $n \times p$. Non

appena sarà calcolata la matrice prodotto, un $m+1$ -esimo thread, che era in attesa, provvederà a stampare la matrice risultato. Le matrici devono essere allocate dinamicamente.

Usare i semafori Posix basati su nome.

8. **ESERCIZIO_08.** Scrivere un programma C, secondo lo standard Posix, in ambiente Linux, che realizzi il prodotto di due matrici di dimensioni $n \times m$ e $m \times p$, in maniera concorrente, utilizzando n thread. Un thread $n+1$ -esimo attenderà il calcolo della matrice prodotto per poi stamparne il contenuto. NB: La traccia non richiede esplicitamente un meccanismo di sincronizzazione. Userò i mutex e variabili di cond.
9. **ESERCIZIO_09.** Si realizzi un programma in C e Posix sotto Linux che, con l'ausilio della libreria Pthread, lancia k thread per calcolare il prodotto di due matrici di dimensione $k \times m$ e $m \times p$. Non appena sarà calcolata la matrice prodotto, un $k+1$ -esimo thread aggiuntivo, che era rimasto in attesa, provvederà a stampare la matrice risultato. Le matrici devono essere allocate dinamicamente. Usare le variabili di condizione.
10. **ESERCIZIO_10.** Si realizzi un programma in C e Posix sotto Linux che, con l'ausilio della libreria Pthread, lancia m thread per calcolare la somma dei prodotti delle colonne di due matrici $m \times m$. L' i -esimo thread, dopo aver calcolato la somma dei prodotti delle colonne i -esime delle due matrici, inserisce il risultato in un array m -dimensionale, in modo concorrente, nella prima locazione libera disponibile. Non appena l'array sarà riempito, un $m+1$ -esimo thread, che era in attesa, provvederà a stamparne il contenuto. Le matrici devono essere allocate dinamicamente. Usare mutex e variabili di condizione.
11. **ESERCIZIO_11.** Si realizzi un programma in C e Posix sotto Linux che, con l'ausilio della libreria Pthread, lancia m thread concorrenti per cercare il massimo di ciascuna delle m righe di una matrice $m \times n$ di interi e scriverlo in un array di dimensione m nella prima posizione libera disponibile. Un thread $m+1$ -esimo, completato il lavoro dei primi m thread, provvede a cercare il massimo dei massimi e a stamparlo. Usare semafori Posix basati su nome e variabili di condizione. La dimensione della matrice può essere fornita in input al programma in fase di esecuzione o da riga di comando.
12. **ESERCIZIO_12.** Scrivere un programma C, secondo lo standard Posix, in ambiente Linux, che realizzi il prodotto di due matrici di dimensioni $n \times m$ e $m \times p$ in maniera concorrente, utilizzando n thread. Un thread $n+1$ esimo attenderà il calcolo della matrice prodotto per stampare il contenuto. Utilizzare i semafori Posix.